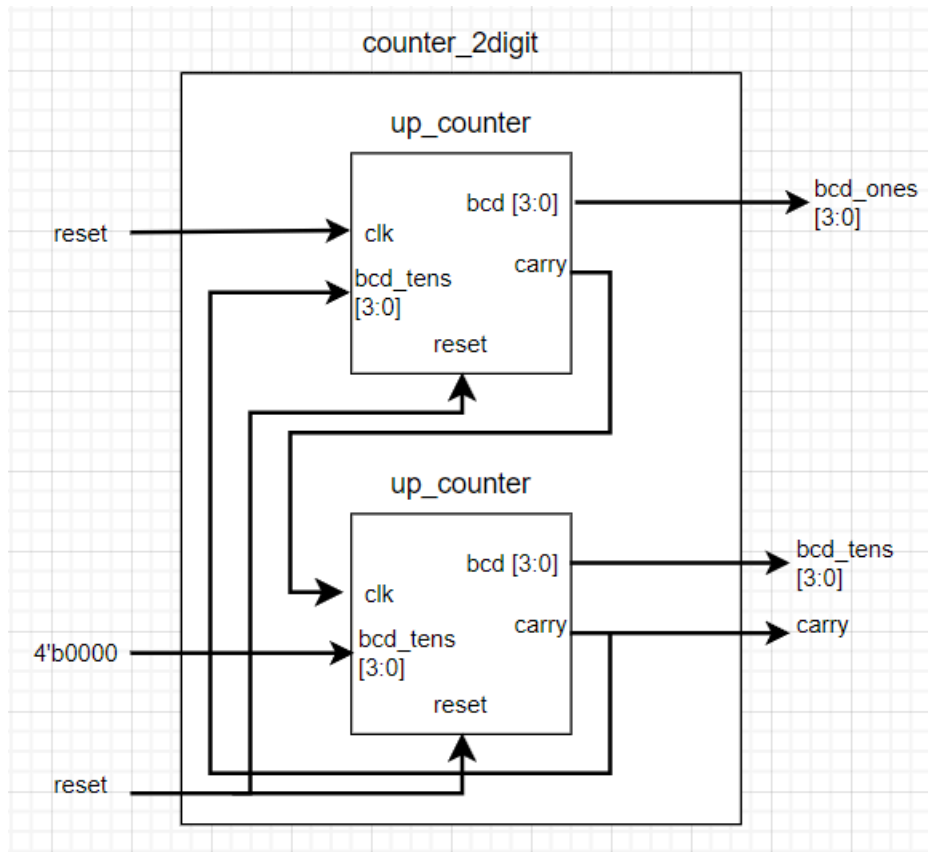


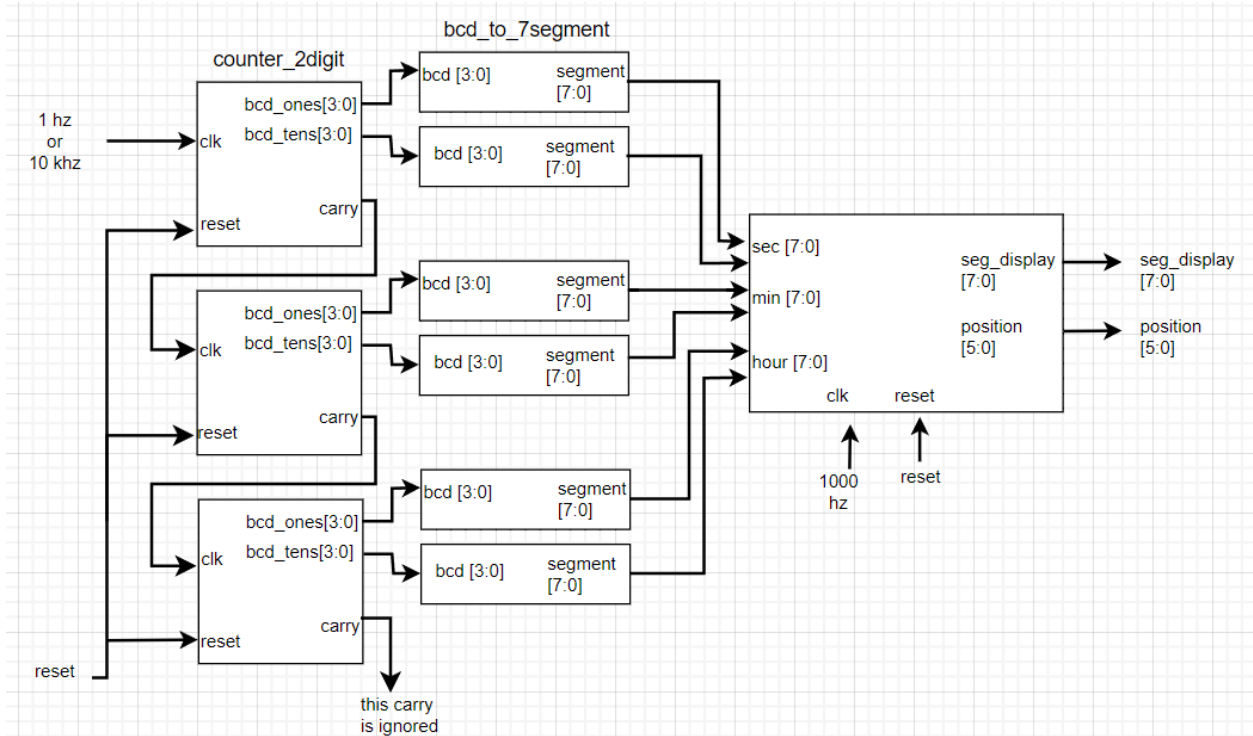
Digital Logic Design – Lab 6

1. **Topic:** 24 Hour Clock Display
2. **Design structure:**

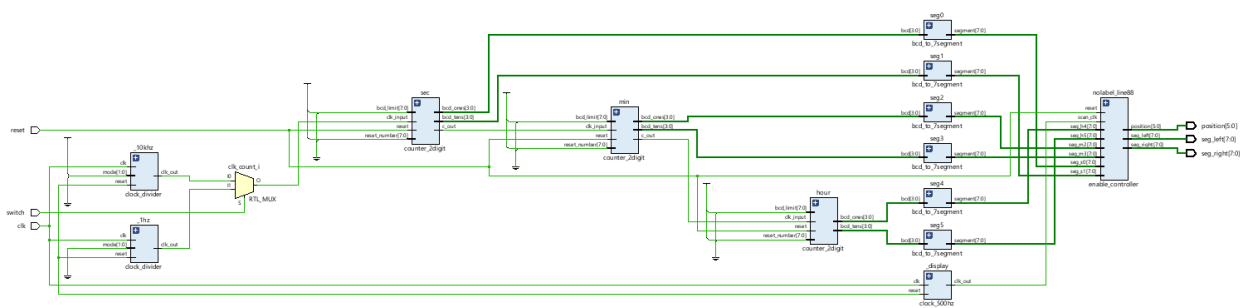
This is the inner workings of the counter_2digit module, which uses an up-counter that has several modifications. For example, the ones up-counter must keep track of the tens up-counter, so the carry of the tens must be introduced into the ones up-counter, and that's the reason the tens up-counter receives a 4'b0000 as input (because it needs no info). This is to help loop back from 23:59:59 to 00:00:00



And this is the whole implementation of the 24 hour display



Also, this is the schematic design generated by Vivado, it is also included in the .zip file



3. **Code:** the code is also in the same .zip file

- **bcd_to_7segment.v**

```

23 // use the bcd count to display the appropriate 7 segment LEDs
24 module bcd_to_7segment(bcd, segment);
25     input [3:0] bcd;
26     output reg [7:0] segment;
27
28     always @(bcd) begin
29         case(bcd)
30             4'b0000 : segment <= 8'b11111100; // 0
31             4'b0001 : segment <= 8'b01100000; // 1
32             4'b0010 : segment <= 8'b11011010; // 2
33             4'b0011 : segment <= 8'b11110010; // 3
34             4'b0100 : segment <= 8'b01100110; // 4
35             4'b0101 : segment <= 8'b10110110; // 5
36             4'b0110 : segment <= 8'b10111110; // 6
37             4'b0111 : segment <= 8'b11100000; // 7
38             4'b1000 : segment <= 8'b11111110; // 8
39             4'b1001 : segment <= 8'b11110110; // 9
40             default : segment <= 8'b01101110; // H
41         endcase
42     end
43 endmodule

```

● clock_divider.v

```

23 // clock divider that takes a 100 MHz, and outputs a modified clock according to the mode:
24 // mode -> 0 for 1hz, 1 for 10khz
25 module clock_divider(clk, mode, reset, clk_out);
26     input clk, reset;
27     input [1:0] mode;
28     output reg clk_out;
29
30     // divider
31     integer counter;
32     always @(posedge clk or posedge reset) begin
33         if(reset)
34             counter <= 0;
35         else if(mode == 1'b0) begin // 1hz
36             if(counter == (100000000 - 2))
37                 counter <= 0;
38             else
39                 counter <= counter + 1;
40         end
41         else if(mode == 1'b1) begin // 10khz
42             if(counter == (10000 - 2))
43                 counter <= 0;
44             else
45                 counter <= counter + 1;
46         end
47     end
48
49     // signal generation
50     always @(posedge clk or posedge reset) begin
51         if(reset)
52             clk_out <= 1'b0;
53         else if(mode == 1'b0) begin // 1hz
54             if(counter < (100000000 / 2))
55                 clk_out <= 1'b1;
56             else
57                 clk_out <= 1'b0;
58         end
59         else if(mode == 1'b1) begin // 10khz
60             if(counter < (10000 / 2))
61                 clk_out <= 1'b1;
62             else
63                 clk_out <= 1'b0;
64         end
65     end
66
67 endmodule

```

```
70 // clock divider that takes a 100 MHz and outputs a 500 Hz clock
```

```
71 module clock_500hz(clk, reset, clk_out);
```

```
72     input clk, reset;
```

```
73     output reg clk_out;
```

```
74     :
```

```
75     // divider
```

```
76     integer counter;
```

```
77     always @(posedge clk or posedge reset) begin
```

```
78         if(reset)
```

```
79             counter <= 0;
```

```
80         else begin
```

```
81             if(counter == (200000 - 2)) // 500 Hz
```

```
82                 counter <= 0;
```

```
83             else
```

```
84                 counter <= counter + 1;
```

```
85         end
```

```
86     end
```

```
87     :
```

```
88     // signal generation
```

```
89     always @(posedge clk or posedge reset) begin
```

```
90         if(reset)
```

```
91             clk_out <= 1'b0;
```

```
92         else begin
```

```
93             if(counter < (200000 / 2)) // 500 Hz
```

```
94                 clk_out <= 1'b1;
```

```
95             else
```

```
96                 clk_out <= 1'b0;
```

```
97         end
```

```
98     end
```

```
99     :
```

```
100 endmodule
```

● counter_2digit.v

```
23 // counter to display the appropriate up-count for the hour, minutes, or seconds given
```

```
24 // a number-reset and a up-count-limit
```

```
25 module counter_2digit(clk_input, reset, reset_number, bcd_limit, c_out, bcd_tens, bcd_ones);
```

```
26     input clk_input, reset;
```

```
27     input [7:0] reset_number, bcd_limit; // [7:4] limit tens, and [3:0] limit ones
```

```
28     :
```

```
29     output c_out;
```

```
30     output [3:0] bcd_tens, bcd_ones;
```

```
31     :
```

```
32     wire carry_ones;
```

```
33     :
```

```
34     // determine up-count display
```

```
35     up_counter count_ones (clk_input, reset, reset_number[3:0], bcd_limit[3:0], bcd_tens, bcd_ones, carry_ones);
```

```
36     up_counter count_tens (carry_ones, reset, reset_number[7:4], bcd_limit[7:4], 4'b0000, bcd_tens, c_out);
```

```
37     :
```

```
38 endmodule
```

```

41 | // up counter that goes from 0 to 9 automatically because of the clock
42 | module up_counter(clk, reset, reset_number, limit, bcd_tens, bcd, carry);
43 |     input clk, reset;
44 |     input [3:0] reset_number, limit, bcd_tens;
45 |
46 |     output reg [3:0] bcd; // count 0~9
47 |     output reg carry; // to carry to tens
48 |
49 |     always @(posedge clk or posedge reset) begin
50 |         if(reset) begin
51 |             carry <= 1'b0;
52 |             bcd <= reset_number;
53 |         end
54 |         else if((limit == 4'b0011) && (bcd == 4'b1001) && (bcd_tens < 4'b0010)) begin
55 |             carry <= 1'b1;
56 |             bcd <= 4'b0000;
57 |         end
58 |         else if((limit == 4'b0011) && (bcd == 4'b0011) && (bcd_tens == 4'b0010)) begin
59 |             carry <= 1'b1;
60 |             bcd <= 4'b0000;
61 |         end
62 |
63 |         else if((limit != 4'b0011) && (bcd >= limit)) begin // limit != 3
64 |             carry <= 1'b1;
65 |             bcd <= 4'b0000;
66 |         end
67 |         else begin
68 |             carry <= 1'b0;
69 |             bcd <= bcd + 4'b0001;
70 |         end
71 |     end
72 | endmodule

```

- **display_24h.v**

```

23 | // module to combine all other modules into one to display a 24hs clock
24 | module display_24h(clk, switch, reset, seg_left, seg_right, position);
25 |     input clk, switch, reset;
26 |
27 |     output [7:0] seg_left, seg_right;
28 |     output [5:0] position;
29 |
30 |     // REMEMBER THAT ...
31 |     //
32 |     // mode -> 0 for 1hz, 1 for 10khz
33 |     // clock_divider(clk, mode, reset, clk_out);
34 |     // clock_500hz(clk, reset, clk_out);
35 |     //
36 |     // bcd_to_7segment(bcd, segment);
37 |     //
38 |     // counter_2digit(clk_input, reset, reset_number, bcd_limit, c_out, bcd_tens, bcd_ones);
39 |     //
40 |     // enable_controller(scan_clk, reset, seg_h5, seg_h4, seg_m3, seg_m2, seg_s1, seg_s0, seg_left, seg_right, position);

```

```

43 // Initializing clocks
44 // mode -> 0 for 1hz, 1 for 1000hz
45 wire clk_1hz, clk_10khz, clk_display;
46 clock_divider_1hz(clk, 0'b0, rst, clk_1hz); // 1 hz
47 clock_divider_10khz(clk, 0'b1, rst, clk_10khz); // 10 khz
48 clock_500hz_display(clk, rst, clk_display); // 500 hz
49
50 reg clk_count;
51 always @(switch) begin
52     if(switch) begin
53         clk_count <= clk_10khz;
54     end
55     else begin
56         clk_count <= clk_1hz;
57     end
58 end
59
60 // second, minute, and hour limit
61 wire [7:0] sec_min_limit = 8'b01011001;
62 wire [7:0] hour_limit = 8'b00100011;
63
64 // reset value for second, minute, and hour
65 wire [7:0] sec_reset = 8'b01000101;
66 wire [7:0] min_reset = 8'b01011001;
67
68 wire [7:0] hour_reset = 8'b00100011;
69
70 wire carry_sec, carry_min, carry_hour;
71
72 wire [3:0] bcd_sec_tens, bcd_sec_ones;
73 wire [3:0] bcd_min_tens, bcd_min_ones;
74 wire [3:0] bcd_hour_tens, bcd_hour_ones;
75
76 // counter for second, minutes, and hour
77 counter_2digit sec (clk_count, reset, sec_reset, sec_min_limit, carry_sec, bcd_sec_tens, bcd_sec_ones);
78 counter_2digit min (carry_sec, reset, min_reset, sec_min_limit, carry_min, bcd_min_tens, bcd_min_ones);
79 counter_2digit hour (carry_min, reset, hour_reset, hour_limit, carry_hour, bcd_hour_tens, bcd_hour_ones);
80
81 wire [7:0] seg_h5, seg_h4, seg_m3, seg_m2, seg_s1, seg_s0;
82 bcd_to_7segment seg5 (bcd_hour_tens, seg_h5);
83 bcd_to_7segment seg4 (bcd_hour_ones, seg_h4);
84 bcd_to_7segment seg3 (bcd_min_tens, seg_m3);
85 bcd_to_7segment seg2 (bcd_min_ones, seg_m2);
86 bcd_to_7segment seg1 (bcd_sec_tens, seg_s1);
87 bcd_to_7segment seg0 (bcd_sec_ones, seg_s0);
88
89 enable_controller(clk_display, reset, seg_h5, seg_h4, seg_m3, seg_m2, seg_s1, seg_s0, seg_left, seg_right, position);
90 endmodule

```

- enable_controller.v

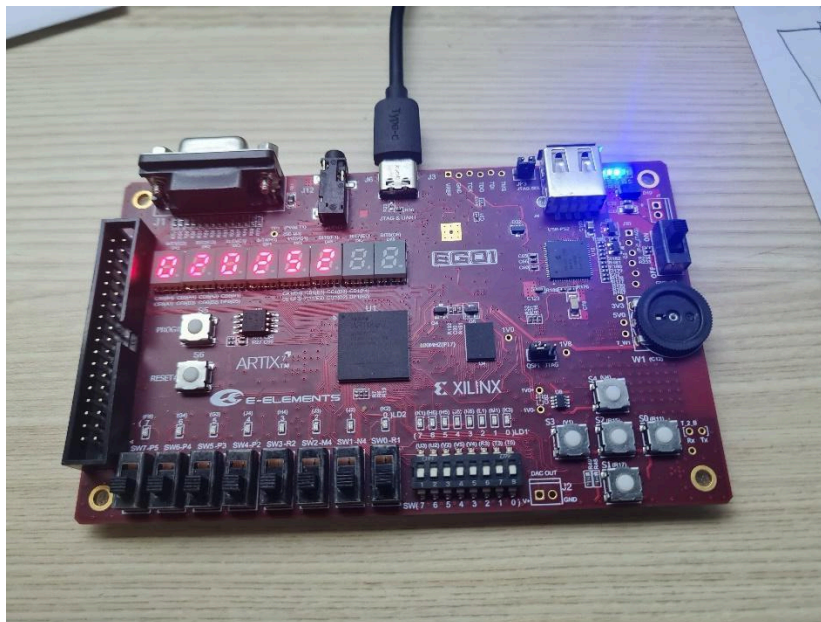
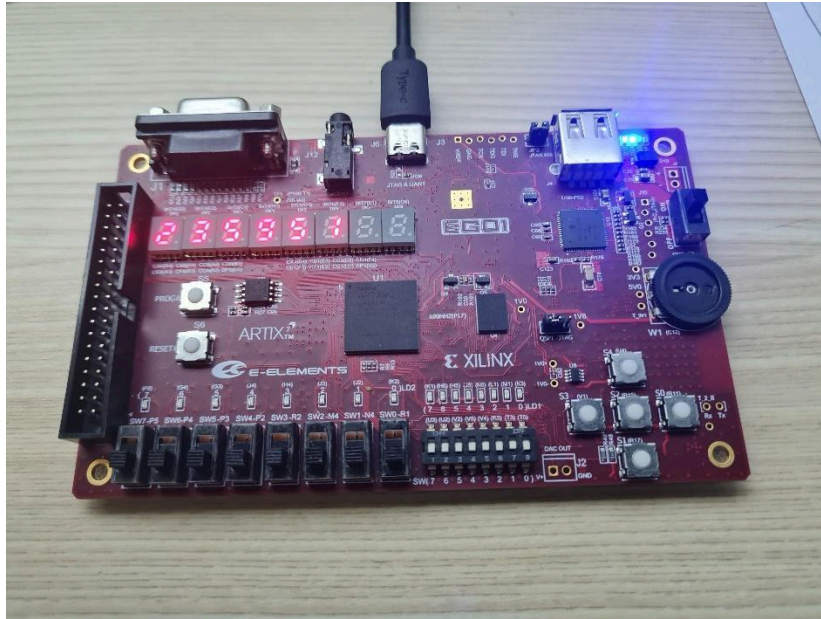
```

23 // this module allows the display of 6 different LEDs at the same time. This is done
24 // by lighting up one LED, and then the other ones, at fast velocities, faster than the
25 // naked eye can process, so we see as if 6 LEDs are light up at the same time
26 module enable_controller(scan_clk, reset, seg_h5, seg_h4, seg_m3, seg_m2, seg_s1, seg_s0, seg_left, seg_right, position);
27     input scan_clk, reset;
28     input [7:0] seg_h5, seg_h4, seg_m3, seg_m2, seg_s1, seg_s0;
29
30     output reg [7:0] seg_left, seg_right;
31     output reg [5:0] position;
32
33     reg [2:0] count;
34     always @(posedge scan_clk or posedge reset) begin
35         if(reset) begin
36             seg_left <= seg_s0;
37             seg_right <= seg_s0;
38             count <= 3'b000;
39             position <= 6'b000001;
40         end
41         else if(count == 3'b000) begin // 0
42             seg_left <= seg_s0;
43             seg_right <= seg_s0;
44             count <= count + 3'b001;
45             position <= 6'b000001;
46         end
47         else if(count == 3'b001) begin // 1
48             seg_left <= seg_s1;
49             seg_right <= seg_s1;
50             count <= count + 3'b001;
51             position <= 6'b000010;
52         end
53         else if(count == 3'b010) begin // 2
54             seg_left <= seg_m2;
55             seg_right <= seg_m2;
56             count <= count + 3'b001;
57             position <= 6'b000100;
58         end
59         else if(count == 3'b011) begin // 3
60             seg_left <= seg_m3;
61             seg_right <= seg_m3;
62             count <= count + 3'b001;
63             position <= 6'b001000;
64         end
65         else if(count == 3'b100) begin // 4
66             seg_left <= seg_h4;
67             seg_right <= seg_h4;
68             count <= count + 3'b001;
69             position <= 6'b010000;
70         end
71         else if(count == 3'b101) begin // 5
72             seg_left <= seg_h5;
73             seg_right <= seg_h5;
74             count <= 3'b000;
75             position <= 6'b100000;
76         end
77     end
78
79 endmodule

```

4. Result: the demo of the lab is in the following youtube video

<https://www.youtube.com/watch?v=3RHR43mAUFU>. The demo displays a clock of 24 hs, that is reset at value 23:59:45. After the clock surpasses the value of 23:59:59, it goes to 00:00:00 and continues counting up. The clock has 2 modes of frequency, a 1 hz clock, and a 10 khz clock, the modes are controlled by a switch in pin R1



5. Reflection

In this lab, we manage to design a clock 24hs system, which involved the use of different clocks at different frequencies. Also, we had to set up different 2 counters with different reset times, and up-counting limits. The counter for seconds, resets at 45, and limits at 59. The counter for minutes, resets at 59, and limits at 59. The counter for hours, resets at 23, and limits at 23. This was a good experience, because to display a correct view, we have to choose an appropriate frequency for display, which I found 500 Hz to be the best one, because 1000 Hz was too fast and caused ghosting